B+tree 구현 Report

과제 개요

이 과제는 데이터베이스 내 효율적인 인덱스 탐색을 위한 B+ tree를 구현하는 과제다. B+ tree의 각 인덱스는 key와 value로 구성되고, key 값 오름차순으로 정렬되어 있다. B+ tree 인덱스의 검색, 범위 검색, 삽입, 삭제의 기능을 구현해야 한다.

내부 노드는 key의 개수 *m*, <key, 왼쪽 자식 노드> 짝의 배열 *p*, 오른쪽 자식 노드를 가리키는 포인터 *r*로 이루어져 있다. 리프 노드는 key의 개수 *m*, <key, value> 짝의 배열 *p*, 오른쪽 형제 노드를 가리키는 포인터 *r*로 이루어져 있다.

또한 command-line 인터페이스를 지원해야 한다. Command-line 입력을 통해 데이터 파일 생성, 입력 파일 삽입, 입력 파일 삭제, key 검색 및 범위 검색을 지원한다.

구현 과정

먼저 B+ tree의 노드 구조와 파일 저장 방식을 정의하였다.

리프 노드

```
{
    "leaf": True,
    "keys": [10, 20, 30],
    "values": [100, 200, 300],
    "next": 4
}
```

- `keys[i]`와 `values[i]`가 하나의 쌍이다.
- `next`는 오른쪽 형제 노드 ID이다.
- 마지막 리프의 `next`는 `None`이다.

내부 노드

```
{
    "leaf": False,
    "keys": [30, 70],
    "children": [1, 4, 7]
}
```

- `children`은 해당 내부 노드의 자식 노드 ID 목록이다.
- `keys`는 자식보다 하나 적다.
- `keys[i]`는 `children[i+1]`의 최솟값이다.

파일 저장 방식

```
{
    "order": 4,
    "root": 8,
    "next_id": 9,
    "nodes": {
        "1": {...},
        "4": {...},
        "7": {...},
        "8": {...}
    }
}
```

위의 정보를 JSON 파일로 저장한다.

- `order`: 노드 최대 자식 수 `b`
- `root`: 루트 노드 ID
- `next_id`: 새 노드에 부여할 ID
- `nodes`: 실제 모든 노드

```
def dump(self) -> dict:
        return {"order": self.order, "root": self.root, "ne
xt_id": self.next_id, "nodes": self.nodes}
```

```
def load_tree(index_file: str) -> BPlusTree:
    try:
        with open(index_file, encoding="utf-8") as stream:
            return BPlusTree(data=json.load(stream))
    except FileNotFoundError as error:
        raise ValueError(f"index file not found: {index_fil
e}") from error
    except (json.JSONDecodeError, KeyError, TypeError) as e
rror:
        raise ValueError(f"invalid index file: {index_fil
e}") from error


def save_tree(tree: BPlusTree, index_file: str) -> None:
    with open(index_file, "w", encoding="utf-8") as stream:
        json.dump(tree.dump(), stream, separators=(",",
":"))
```

`dump()`로 저장할 dictionary를 만들고, `load_tree()`와 `save_tree()`로 JSON에 접근하여 읽고 쓴다. JSON 객체의 key는 string으로 저장되므로 로드 시 노드 ID를 `int`로 다시 바꿨다.

노드 Constraints

노드의 다음의 특성(property)들을 정의하였다.

```
@property
def max_keys(self) -> int:
    return self.order - 1

@property
def min_leaf_keys(self) -> int:
    return (self.max_keys + 1) // 2

@property
def min_internal_children(self) -> int:
    return (self.order + 1) // 2
```

`b = 4`일때:

- 리프 최대 key의 개수: 4 - 1 = 3
- 리프 최소 key의 개수: (3 + 1) // 2 = 2
- 내부 노드 최대 자식 수: 4
- 내부 노드 최소 자식 수: (3 + 1) // 2 = 2

내부 노드 최신 상태 유지

```
def _minimum(self, node_id: int) -> int:
        node = self.nodes[node_id]
        while not node["leaf"]:
            node = self.nodes[node["children"][0]]
        return node["keys"][0]
```

`_minimum()` 함수는 트리의 최솟값을 찾는 함수다. 리프에 도달할 때까지 첫번째 자식 (`children[0]`)으로 내려가고, 리프에 도달하면 최솟값인 첫번째 key를 반환한다.

```
def _refresh(self, node_id: int) -> None:
        node = self.nodes[node_id]
        if not node["leaf"]:
            node["keys"] = [self._minimum(child) for child
in node["children"][1:]]
```

`_refresh()` 함수는 내부 노드의 key 목록을 다시 계산한다.

```
def _child_index(self, node: dict, key: int) -> int:
        return bisect.bisect_right(node["keys"], key)
```

`_child_index()` 함수는 내부 노드에서 어느 자식으로 갈지 결정한다. `bisect_right()` 를 사용하면 이분 시에 대상이 되는 key 값과 일치하더라도 하나 뒤, 즉 오른쪽 자식을 가리키기 때문에 적절한 자식을 선택할 수 있다.

단일 key 검색

```
def search(self, key: int) -> tuple[list[list[int]], int |
None]:
        node = self.nodes[self.root]
        path: list[list[int]] = []
```

```
        while not node["leaf"]:
            path.append(list(node["keys"]))
            node = self.nodes[node["children"][self._child_
index(node, key)]]
        pos = bisect.bisect_left(node["keys"], key)
        value = node["values"][pos] if pos < len(node["key
s"]) and node["keys"][pos] == key else None
        return path, value
```

루트에서 시작해 리프까지 내려가며 탐색하는 방식이다. 탐색 과정에서 `path`에 내부 노드의 key 목록을 기록한다. `_child_index()`로 적절한 자식을 선택하고, 리프 노드에 도달하면 `bisect_left()`로 대상 key의 위치를 찾는다. 해당 `pos`가 검색하고자 하는 key와 같으면 value를 반환한다.

범위 검색

```
def range_search(self, start: int, end: int) -> list[tuple
[int, int]]:
        if start > end:
            return []
        node = self.nodes[self.root]
        while not node["leaf"]:
            node = self.nodes[node["children"][self._child_
index(node, start)]]
        pos = bisect.bisect_left(node["keys"], start)
        result = []
        while True:
            while pos < len(node["keys"]):
                key = node["keys"][pos]
                if key > end:
                    return result
                result.append((key, node["values"][pos]))
                pos += 1
            if node["next"] is None:
                return result
            node = self.nodes[node["next"]]
            pos = 0
```

루트부터 시작해 범위 시작 값이 있어야 할 리프 노드를 찾는다. 해당 리프에서 범위 시작 값 **이상**인 첫번째 key를 `bisect_left()`로 찾는다. 현재 리프의 key-value 쌍을 `result`에 추가하고, 현재 리프를 모두 읽으면 `next`로 오른쪽 리프로 이동한다. key 값이 범위 끝 값보다 커지면 종료한다.

삽입

```
def insert(self, key: int, value: int) -> bool:
        """Insert a pair. Returns False when the key alread
y exists."""
        split = self._insert(self.root, key, value)
        if split is None:
            return True
        old_root = self.root
        self.root = self._new_internal([old_root, split])
        return True

def _insert(self, node_id: int, key: int, value: int) -> in
t | None:
        node = self.nodes[node_id]
        if node["leaf"]:
            pos = bisect.bisect_left(node["keys"], key)
            if pos < len(node["keys"]) and node["keys"][po
s] == key:
                raise KeyError(key)
            node["keys"].insert(pos, key)
            node["values"].insert(pos, value)
            if len(node["keys"]) <= self.max_keys:
                return None
            return self._split_leaf(node_id)

        child_pos = self._child_index(node, key)
        try:
            split = self._insert(node["children"][child_po
s], key, value)
        except KeyError:
            raise
        if split is not None:
            node["children"].insert(child_pos + 1, split)
```

```
        self._refresh(node_id)
        if len(node["children"]) <= self.order:
            return None
        return self._split_internal(node_id)
```

삽입은 `_insert()`가 재귀적으로 수행한다.

리프에 삽입

`bisect_left()`로 삽입할 위치인 `pos`를 찾고, `keys`와 `values`의 같은 위치에 새 데이터를 삽입한다. 이때 `pos`에 해당하는 key가 이미 존재하면 `KeyError`를 발생시켜 중복 삽입을 막는다. 최대 key의 개수보다 커지면 반환하여 종료하고, 리프가 넘치면 리프를 분할한다.

리프 분할

```
def _split_leaf(self, node_id: int) -> int:
        left = self.nodes[node_id]
        right_id = self._new_leaf()
        right = self.nodes[right_id]
        cut = (len(left["keys"]) + 1) // 2
        right["keys"] = left["keys"][cut:]
        right["values"] = left["values"][cut:]
        left["keys"] = left["keys"][:cut]
        left["values"] = left["values"][:cut]
        right["next"] = left["next"]
        left["next"] = right_id
        return right_id
```

`_split_leaf()`는 오른쪽 리프를 새로 만들고, `cut`으로 절반을 자른 뒤 뒤쪽 절반을 새로 만든 리프로 옮긴다. 기존의 `right`가 기존의 `next`를 가리키게 하고, `left`가 새로 만든 오른쪽 리프를 가리키도록 리프 연결을 수정하였다.

```
            ...
            if split is not None:
                node["children"].insert(child_pos + 1, split)
        self._refresh(node_id)
        ...
```

`_split_leaf()`가 호출되어 자식이 분할되면 새로 만든 오른쪽 노드의 ID가 반환값이다. 부모는 이 반환값을 기존 자식 바로 다음인 `child_pos + 1` 위치에 삽입한다.

내부 노드 분할

```
def _split_internal(self, node_id: int) -> int:
        left = self.nodes[node_id]
        cut = len(left["children"]) // 2
        right_children = left["children"][cut:]
        left["children"] = left["children"][:cut]
        right_id = self._new_internal(right_children)
        self._refresh(node_id)
        return right_id
```

내부 노드의 자식의 수가 `b`를 초과하면 `_split_internal()`이 실행된다. 여기서는 `_split_leaf()`와 다르게 key가 아닌 자식 포인터 목록을 반으로 나눈다.

루트 노드 분할

```
def insert(self, key: int, value: int) -> bool:
        """Insert a pair. Returns False when the key alread
y exists."""
        split = self._insert(self.root, key, value)
        if split is None:
            return True
        old_root = self.root
        self.root = self._new_internal([old_root, split])
        return True
```

기존 루트가 분할되면 더 높은 새 루트를 만들어야 한다. `_insert()`의 반환값이 있으면 `old_root`와 새 오른쪽 노드를 자식으로 갖는 새 내부 루트 `self.root`를 만든다.

삭제

```
def delete(self, key: int) -> bool:
        if not self._delete(self.root, key, True):
            return False
        while not self.nodes[self.root]["leaf"] and len(sel
```

```
f.nodes[self.root]["children"]) == 1:
            root = self.nodes[self.root]
            old_root = self.root
            self.root = root["children"][0]
            del self.nodes[old_root]
        if not self.nodes[self.root]["leaf"] and not self.n
odes[self.root]["children"]:
            old_root = self.root
            self.root = self._new_leaf()
            del self.nodes[old_root]
        self._relink_leaves()
        return True

def _delete(self, node_id: int, key: int, is_root: bool) ->
bool:
        node = self.nodes[node_id]
        if node["leaf"]:
            pos = bisect.bisect_left(node["keys"], key)
            if pos == len(node["keys"]) or node["keys"][po
s] != key:
                return False
            del node["keys"][pos]
            del node["values"][pos]
            return True

        child_pos = self._child_index(node, key)
        if not self._delete(node["children"][child_pos], ke
y, False):
            return False
        self._repair_child(node_id, child_pos)
        self._refresh(node_id)
        return True
```

삭제 역시 삽입과 마찬가지로 `_delete()`가 삭제를 수행한 뒤, 재귀적으로 부모 방향으로 올라오면서 B+ tree의 균형을 유지한다.

리프에서 삭제

`bisect_left()`로 삭제할 key의 위치를 찾고, `keys`와 `values`의 같은 위치의 노드를 삭제한다. 이때 삭제하고자 하는 key가 없으면 `False`를 반환한다.

재분배: 왼쪽 형제에게서 빌리기

```
def _borrow_left(self, left_id: int, child_id: int) -> Non
e:
        left, child = self.nodes[left_id], self.nodes[child
_id]
        if child["leaf"]:
            child["keys"].insert(0, left["keys"].pop())
            child["values"].insert(0, left["values"].pop())
        else:
            child["children"].insert(0, left["children"].po
p())
```

왼쪽 형제에 여유가 있으면

- 리프: 왼쪽 형제의 `keys`와 `values`에 `pop()`을 수행하여 마지막 <key, value> 쌍을 현재 리프의 맨 앞인 0번째로 옮긴다.
- 내부 노드: 왼쪽 형제의 `children`에 `pop()`을 수행하여 마지막 자식 포인터를 현재 노드의 맨 앞인 0번째로 옮긴다.

재분배: 오른쪽 형제에게서 빌리기

```
def _borrow_right(self, child_id: int, right_id: int) -> No
ne:
        child, right = self.nodes[child_id], self.nodes[rig
ht_id]
        if child["leaf"]:
            child["keys"].append(right["keys"].pop(0))
            child["values"].append(right["values"].pop(0))
        else:
            child["children"].append(right["children"].pop
(0))
```

오른쪽 형제에 여유가 있으면

- 리프: 오른쪽 형제의 `keys`와 `values`에 `pop(0)`을 수행하여 첫번째 <key, value> 쌍을 현재 리프의 맨 뒤로 붙인다.
- 내부 노드: 오른쪽 형제의 `children`에 `pop(0)`을 수행하여 첫번째 자식 포인터를 현재 노드의 맨 뒤로 붙인다.

병합

만약 어느 형제도 여유가 없다면 두 형제를 합친다.

```
def _merge(self, left_id: int, right_id: int) -> None:
        left, right = self.nodes[left_id], self.nodes[right
_id]
        if left["leaf"]:
            left["keys"].extend(right["keys"])
            left["values"].extend(right["values"])
            left["next"] = right["next"]
        else:
            left["children"].extend(right["children"])
            self._refresh(left_id)
```

- 리프: 왼쪽 리프에 오른쪽 형제의 `keys`와 `values`를 이어 붙이고 리프 연결을 수정한다.
- 내부 노드: 두 노드의 자식 포인터를 이어 붙인 뒤 `left_id`를 다시 계산한다.

루트 축소

삭제와 병합을 반복하다보면 루트 내부 노드에 자식이 하나만 남을 수 있다. 이 경우 루트는 불필요하므로 유일한 자식을 새 루트로 설정한다.

```
            ...
            if not root["leaf"] and len(root["children"]) =
= 1:
            old_root = self.root
            self.root = root["children"][0]
            del self.nodes[old_root]
        ...
```

```
            ...
            while not self.nodes[self.root]["leaf"] and len
(self.nodes[self.root]["children"]) == 1:
            root = self.nodes[self.root]
            old_root = self.root
            self.root = root["children"][0]
            del self.nodes[old_root]
        ...
```

`b = 2`인 경우 삭제를 할 때 하나의 내부 노드가 하나의 자식만 갖는 상황이 발생할 수 있다. 따라서 `while` 문을 사용하여 루트 노드를 반복적으로 축소하여 이 상황이 발생하지 않도록 루트가 리프가 되거나 자식이 둘 이상일 때까지 반복한다.

```
def _repair_child(self, parent_id: int, index: int) -> Non
e:
        parent = self.nodes[parent_id]
        child_id = parent["children"][index]
        child = self.nodes[child_id]
        if child["leaf"]:
            minimum = self.min_leaf_keys
            size = len(child["keys"])
        else:
            minimum = self.min_internal_children
            size = 0 if self._subtree_empty(child_id) else
len(child["children"])
        if size >= minimum:
            return
        if not child["leaf"] and self._subtree_empty(child_
id):
            del parent["children"][index]
            self._drop_subtree(child_id)
            return
        ...
```

```
def _subtree_empty(self, node_id: int) -> bool:
        node = self.nodes[node_id]
        while not node["leaf"]:
```

```
            if not node["children"]:
                return True
            node = self.nodes[node["children"][0]]
        return not node["keys"]
```

추가로 내부 노드 아래 리프의 key가 비어 있는 경우가 발생할 수 있어 `_repair_child()`에서 `_subtree_empty()`를 통해 key가 비어있는지 확인한다. `_subtree_empty()` 함수는 첫번째 자식을 따라 리프까지 내려가 key가 비어있는지 확인한다. 내부 노드의 자식 목록 자체가 비어있는 경우도 빈 서브트리로 판별한다.

```
if not child["leaf"] and self._subtree_empty(child_id):
    del parent["children"][index]
    self._drop_subtree(child_id)
    return
```

빈 서브트리는 병합하지 않고 자식 목록에서 제거한다. 병합하지 않는 이유는 빈 서브트리를 다른 노드에 합치면 비어 있는 리프가 오른쪽 자식으로 남아 `_minimum()` 계산 시 빈 리프의 첫 key를 잘못 찾을 수 있기 때문이다. 따라서 `_drop_subtree()` 함수로 빈 내부 노드와 그 아래 빈 리프를 재귀적으로 모두 제거하였다.

```
if not self.nodes[self.root]["leaf"] and not self.nodes[sel
f.root]["children"]:
    old_root = self.root
    self.root = self._new_leaf()
    del self.nodes[old_root]
```

루트의 자식 목록이 모두 비어있는 경우도 처리하였다.

CSV 처리 & CLI 지원

```
def read_insert_csv(filename: str):
    with open(filename, newline="", encoding="utf-8") as st
ream:
        for line_number, row in enumerate(csv.reader(strea
m), 1):
            if len(row) != 2:
                raise ValueError(f"{filename}:{line_numbe
r}: expected key,value")
            yield int(row[0]), int(row[1])
```

```
def read_delete_csv(filename: str):
    with open(filename, newline="", encoding="utf-8") as st
ream:
        for line_number, row in enumerate(csv.reader(strea
m), 1):
            if len(row) != 1:
                raise ValueError(f"{filename}:{line_numbe
r}: expected one key")
            yield int(row[0])
```

`read_insert_csv()`는 삽입하고자 하는 CSV 파일의 각 행을 <`key, value`> 쌍을 읽는다.

`read_delete_csv()`는 삭제하고자 하는 CSV 파일의 각 행을 `key` 하나로 읽는다.

마지막으로 `main()` 함수에는 인자 개수와 옵션을 검사해 5개의 명령어를 처리하는 로직이 구현되어 있다.

컴파일 과정

요구 환경

- OS: Windows 또는 macOS
- Python: Python 3.10 이상
- 외부 패키지: 필요 없음. `bisect`, `csv`, `json`, `sys` 등 Python 표준 라이브러리만 사용한다.

실행 위치

압축을 해제한 뒤 `Source` 디렉터리에서 명령을 실행한다.

```
cd B+tree_Assignment/Source
```

Windows PowerShell과 macOS/Linux Terminal 모두 아래 명령 형식을 사용할 수 있다. Windows에서 Python 명령이 `python`으로 등록되어 있으면 `python3` 대신 `python`을 사용한다.

검증

구현한 B+ tree가 일반적인 상황 이외에 극단적인 상황에서도 정상적으로 작동하는지 확인하기 위해 테스트 코드를 작성하여 검증하였다.

테스트	검증 내용
최소 차수 및 편향 입력	`b = 2`에서 250개 키를 오름차순 삽입하고 내림차순 삭제하여 반복 분할·병합·루트 축소를 검증한다.
다양한 차수와 무작위 연산	`b = 2, 3, 4, 5, 8, 31`에서 음수를 포함한 301개 키의 무작위 삽입과 삭제를 검증한다.
저장 및 재로딩	JSON 파일에 저장한 트리를 다시 읽은 뒤 검색·범위 결과와 구조가 동일한지 확인한다.
범위 경계와 리프 연결	존재하지 않는 시작/끝 키, 음수 키, 여러 리프를 가로지르는 범위를 검증한다.
중복 및 없는 키	중복 삽입과 존재하지 않는 키의 검색·삭제가 기존 데이터를 손상하지 않는지 확인한다.
CLI 종단 간 테스트	`-c`, `-i`, `-d`, `-s`, `-r`의 CSV 처리, 표준 출력 형식, 인덱스 덮어쓰기를 검증한다.